

Depuración Práctica con IA: Debugging Real + Modularización + Pipeline

Ejercicio de Clase · Ejercicio 2 de 3 · Aplicación práctica con IA, modularización y CI/CD

Nombre del alumno/a	Angel Echenique
Fecha	04/04/2026
🔗 Enlace GitHub del repositorio	

📌 **Instrucciones:** Ejercicio práctico: analizarás código con errores reales, construirás el prompt para la IA, interpretarás su respuesta, refactorizarás usando modularización y configurarás el pipeline de CI/CD. Usa tus apuntes de los módulos anteriores.

BLOQUE 1 — Analiza el código con errores

Tienes el siguiente sistema de gestión de pedidos con múltiples errores. Lee el código completo antes de responder:

```
# sistema_pedidos.py — CON ERRORES INTENCIONADOS

import datetime

productos = {
    'cafe': {'precio': 2.5, 'stock': 10},
    'te': {'precio': 2.0, 'stock': 5},
    'zumo': {'precio': 3.0, }, # ←falta 'stock'
}

historial = []

def calcular_total(pedido):
    total = 0
    for item in pedido:
        total = total + productos[item]['precio'] # ←KeyError si no existe
    return total / len(pedido) # ←BUG: debería ser sum, no promedio

def registrar_pedido(cliente, pedido):
    if len(pedido) > 0:
        total = calcular_total(pedido)
```

```

fecha = datetime.date.today
historial.append({
    'cliente': cliente,
    'pedido': pedido,
    'total': total,
    'fecha': fecha
})
print('Pedido de ' + cliente + ' registrado: ' + total) # ←TypeError

def obtener_resumen():
    if historial == []:
        return 'No hay pedidos'
    totales = []
    for registro in historial:
        totales.append(registro['total'])
    media = sum(totales) / len(totales)
    return f'Total pedidos: {len(historial)} | Media: {media}'

# Test
registrar_pedido('Ana', ['cafe', 'te'])
registrar_pedido('Luis', ['zumo', 'cafe'])
print(obtener_resumen())

```

1.1 — Identifica los errores antes de usar la IA

Sin consultar la IA todavía, identifica manualmente los errores que encuentres. Para cada uno indica el tipo (Lógico / Ejecución / Diseño):

#	Línea aprox.	Descripción del error	Tipo (L/E/D)
1	Línea 10 (return total / len(pedido))	El código está calculando la media del precio de los productos en lugar de sumar el total del pedido. Además, si la lista pedido está vacía, dará un error ZeroDivisionError.	L (Lógico) y potencialmente E (Ejecución).
2	Línea 14 (fecha = datetime.date.today)	Falta poner los paréntesis () para llamar a la función. Al no tenerlos, la variable guarda la referencia al método, no la fecha actual.	L (Lógico / Ejecución cuando se intente usar la fecha).
3	Línea 19 (print('Pedido de ' + cliente + ' registrado: ' + total))	Intenta concatenar texto (str) con un número (float). Esto hará que el programa se rompa. Debería usar str(total) o una f-string (f"Pedido... {total}").	E (Ejecución - TypeError).

#	Línea aprox.	Descripción del error	Tipo (L/E/D)
4	Línea 4 ('zumo': {'precio': 3.0})	A este producto le falta la clave 'stock'. Si en el futuro otra función intenta restar el stock al vender un zumo, el programa fallará con un KeyError.	D (Diseño - Estructura de datos inconsistente).
5	Línea 9 (total = total + productos[item]['precio'])	Si un cliente pide un producto que no está en el diccionario productos (por ejemplo, 'agua'), el programa lanzará un KeyError porque no hay ninguna validación previa que compruebe si el ítem existe.	D (Diseño por falta de validación) / E (Ejecución si ocurre).

BLOQUE 2 — Construye el prompt para la IA

Redacta el prompt completo que enviarías a la IA para depurar este código. Recuerda que debe incluir los 5 elementos del prompt profesional vistos en clase (código, error, contexto, entorno, tipo de ayuda):

Esto sería una plantilla

Actua como experto en depuracion.

Estoy usando Python 3.10 (**Entorno**).

Analiza el siguiente código de un sistema de pedidos (**Contexto/Código**). Debería calcular el total de los productos y registrar el historial, pero sospecho que tiene varios errores lógicos y de ejecución (**Comportamiento esperado/Error**).

Realiza un análisis de causa raíz, explícame los errores y devuélveme el código corregido aplicando buenas prácticas (**Tipo de ayuda**).

Aquí está el código: **[PEGAMOS EL CODIGO]**

El prompt que utilizaria

Actúa como experto en depuración de código.

Estoy usando **Python 3.10**.

Analiza el siguiente código de un sistema de gestión de pedidos. El programa utiliza un diccionario de productos y guarda los registros en una lista historial. La función `calcular_total()` debería sumar el precio exacto de los ítems del pedido, y `registrar_pedido()` debería guardar la fecha actual y el total, para finalmente imprimir un mensaje de confirmación por consola.

Sin embargo, el código presenta múltiples fallos que necesito solucionar: hay un error lógico al calcular el total (parece estar calculando un promedio en la línea del `return`), un fallo al intentar registrar la fecha con la librería `datetime`, errores de tipado al imprimir el mensaje final y posibles riesgos de `KeyError` con el inventario de productos si falta la clave `'stock'` o si se pide un producto inexistente.

Por favor, realiza un análisis de causa raíz de estos problemas, explícame los errores detalladamente y devuélveme el código corregido aplicando buenas prácticas de validación. Además, realiza una auditoría de seguridad para detectar posibles vulnerabilidades.

Aquí está el código: `[Codigo del sistema_pedidos.py]`"

2.2 — Analiza la respuesta de la IA

□ Respuesta simulada de la IA: He encontrado 4 problemas en tu código: 1. CAUSA RAÍZ — `calcular_total()`: La función divide por `len(pedido)` en lugar de devolver la suma. Es un error lógico: devuelve la media del precio por ítem, no el total del pedido. 2. ERROR DE EJECUCIÓN — `registrar_pedido()`: En la línea `print('Pedido de ' + cliente + ...)` intentas concatenar `str` + `float`. Necesitas convertir `total` con `str(total)` o usar `f-strings`. 3. ERROR DE EJECUCIÓN — `datetime.date.today` falta el `()` para llamar a la función. Sin paréntesis, fecha almacena el método, no el resultado. 4. ERROR DE DISEÑO — El diccionario `'zumo'` no tiene clave `'stock'`. La función `calcular_total()` accedería a `productos[item]['precio']` sin problema, pero cualquier función que use el `stock` fallaría con `KeyError`. SOLUCIÓN RECOMENDADA: Valida la estructura del producto antes de procesarlo y usa `f-strings` para evitar errores de concatenación.

Responde estas preguntas sobre la respuesta de la IA:

Pregunta	Tu respuesta
¿Coinciden los errores que encontraste tú con los que detectó la IA? ¿Qué errores te faltaron o sobraron?	Sí coinciden , la IA detectó los mismos que nosotros (la división errónea, la concatenación, el <code>today()</code> sin paréntesis y la falta de <code>stock</code>). Me faltó agregar el <code>KeyError</code> en caso de pedir un producto inexistente.
¿La IA hizo un Root Cause Analysis (causa raíz) o solo describió el síntoma? Justifica.	Sí . No se limitó a decir "cambia la línea 10". Explicó por qué fallaba (ej. "devuelve la media... no el total" o "falta el <code>()</code> para llamar a la función"). Fue al origen del problema.
¿Qué añadirías al prompt para que la IA también revisara vulnerabilidades de seguridad?	Realiza una auditoría de seguridad del código basándote en OWASP Top 10 y detecta posibles vulnerabilidades o exposiciones de datos.
¿Confiarías en aplicar los cambios sin revisarlos? ¿Por qué sí o por qué no?	No . Como vimos en clase, la IA es un asistente, no un reemplazo. Siempre debemos entender los cambios, validar que no rompan otra parte del sistema y, sobre todo, escribir tests unitarios para comprobar que la lógica ahora es correcta.

BLOQUE 3 — Refactoriza con Modularización (conexión con Módulo anterior)

□ Conexión con el módulo de Modularización: una vez corregidos los errores con ayuda de la IA, el siguiente paso profesional es refactorizar el código en módulos separados con responsabilidad única.

3.1 — Diseña la estructura modular

El código sistema_pedidos.py mezcla 3 responsabilidades. Define la estructura de archivos modular que crearías:

Módulo (nombre archivo)	Responsabilidad única	Funciones que incluiría
datos.py	Almacenar y gestionar los datos del sistema (simulando una base de datos).	➤ El diccionario productos ➤ La lista historial.
pedidos.py	Contener la lógica de negocio (cálculos y registros).	➤ calcular_total(pedido) ➤ registrar_pedido(cliente, pedido).
main.py (Entrada)	Mostrar la información y estadísticas al usuario o ejecutar el flujo.	➤ obtener_resumen().

3.2 — Prompt para que la IA refactorice

Escribe el prompt que le darías a la IA para que refactorice el código corregido siguiendo los principios de modularización:

Toma el código corregido que me acabas de dar y refactorízalo aplicando el principio de Responsabilidad Única. Separa el código en tres módulos: datos.py (para diccionarios y listas), pedidos.py (para la lógica de cálculo y registro) y reportes.py (para los resúmenes). Asegúrate de incluir los import necesarios en cada archivo para que funcionen entre sí.

BLOQUE 4 — Integración en el Pipeline CI/CD (conexión con Módulo anterior)

□ Conexión con el módulo de Pipeline: una vez tienes el código modularizado y corregido, debes asegurarte de que el pipeline lo valide automáticamente en cada push.

Diseña el fichero `.github/workflows/ci.yml` para este proyecto. Indica qué pasos debe incluir el pipeline en cada fase:

Fase del pipeline	¿Qué debe hacer?	Comando o herramienta
1. Checkout del código	Clona el repositorio en la máquina virtual.	actions/checkout@v3 (o v4)
2. Configurar entorno Python	Instala la versión correcta de Python.	actions/setup-python@v4 (con python-version: '3.10')
3. Instalar dependencias	Descarga las librerías necesarias.	pip install -r requirements.txt
4. Ejecutar linter (calidad)	Revisa que el código cumpla con los estándares PEP8.	flake8 . o pylint.
5. Ejecutar tests unitarios	Valida que la lógica (como sumar el total) funcione.	pytest.
6. Medir cobertura	Asegura que los tests cubren gran parte del código.	pytest-cov (ej. pytest --cov=.)
7. Análisis de seguridad	Busca vulnerabilidades o dependencias inseguras.	bandit -r . o safety check.

BLOQUE 5 — Reflexión integrada

Explica el flujo completo de trabajo profesional que acabas de realizar: desde detectar un error, pasando por usar la IA para depurarlo, refactorizar en módulos y automatizar la validación en el pipeline. ¿Qué aporta cada paso y por qué no se pueden saltar?

Cada paso aporta valor y se protege el uno al otro

- **Detectar e identificar errores** manualmente nos da el contexto inicial y mejora nuestra comprensión del problema.
- **Usar la IA (Debugging)** acelera drásticamente el proceso y nos enseña la causa raíz, actuando como un mentor experto en tiempo real.
- **Refactorizar (Modularización)** prepara nuestro código para el futuro. Al separar responsabilidades, si en el futuro la base de datos cambia, solo tocamos `datos.py` sin romper los cálculos de `pedidos.py`.
- **Pipeline (CI/CD)** es la red de seguridad final. Automatiza las pruebas (tests, linter, seguridad) para garantizar que, si otro desarrollador (o nosotros mismos en el futuro) introduce un nuevo bug o rompe la estructura, el código no llegue a producción.

Ningún paso se puede saltar porque la IA puede equivocarse (necesita nuestra revisión), el código espagueti es inmanejable a largo plazo (necesita modularización) y nosotros olvidamos probar todo a veces y entonces (necesitamos CI/CD).